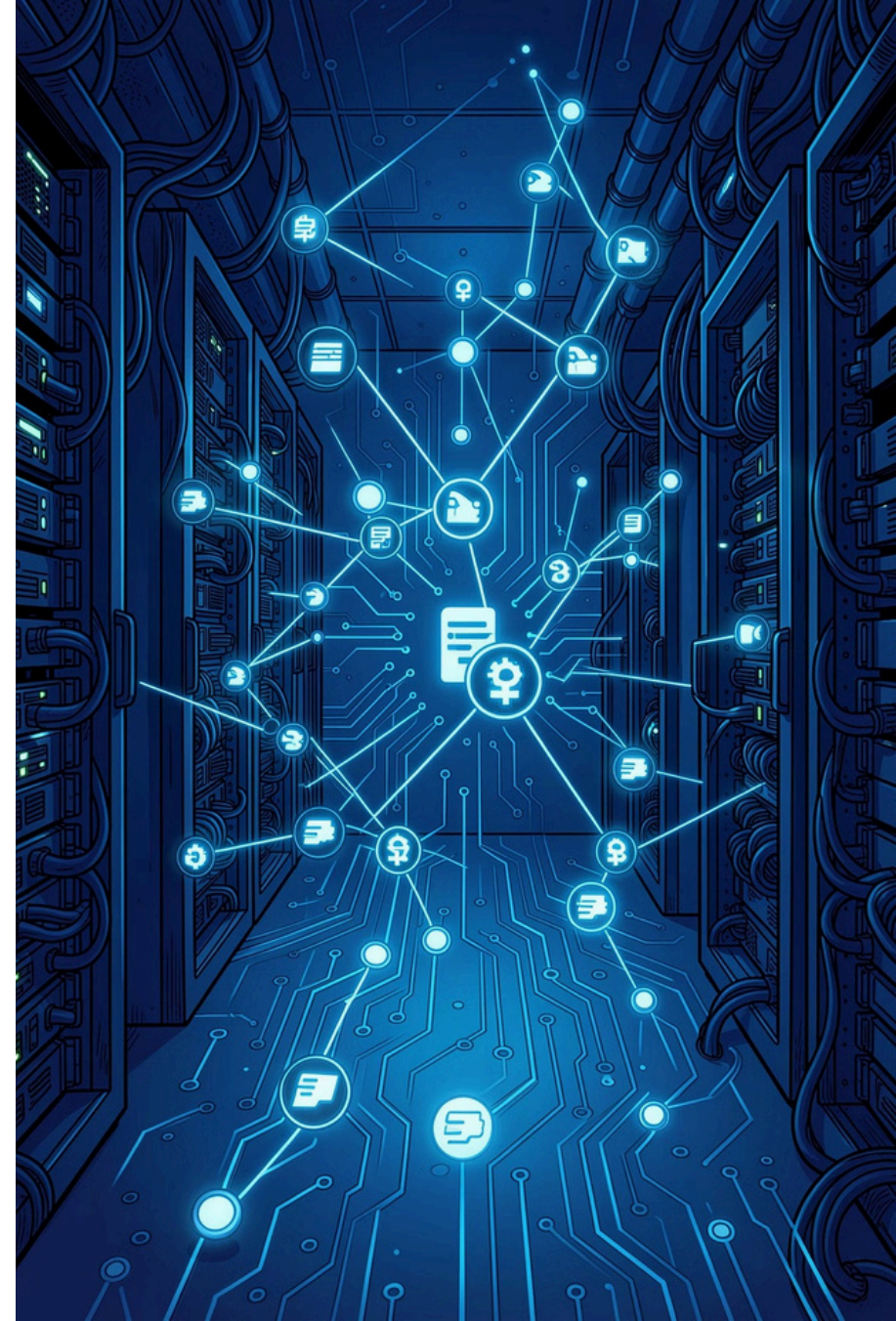


Payment Orchestration Layer

Intelligent multi-gateway routing with failover, idempotency, and webhook reconciliation

Dhivya S B.Tech IT



Problem Statement

Fragile Integrations

Single-gateway failure blocks all checkouts — zero redundancy

No Optimisation

Cost, speed, and success rate never evaluated per transaction

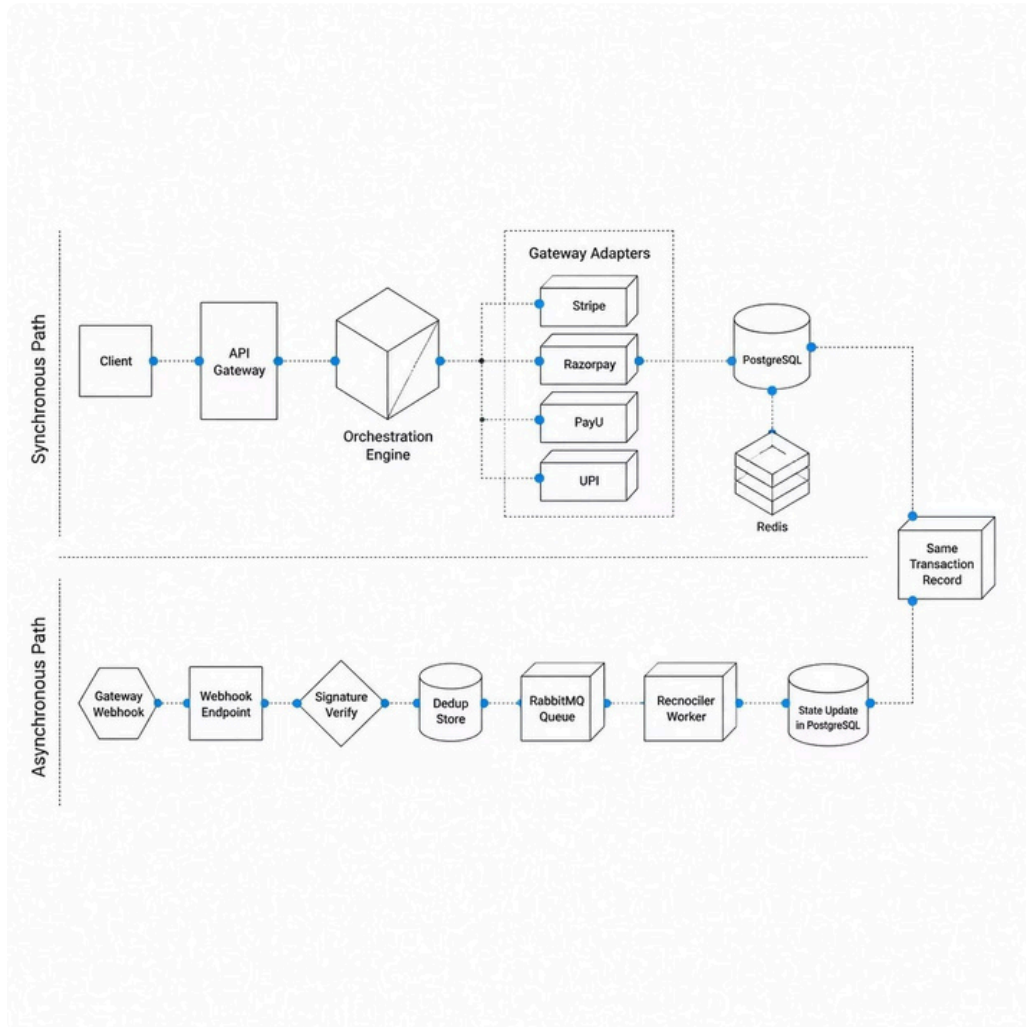
Fragmented Reconciliation

Non-unified state tracking across multiple gateway callbacks

- ❏ **Goal:** Route transactions across Stripe, Razorpay, PayU, and UPI — with failover under 2 seconds.



System Architecture



Two Entry Paths

- **Synchronous:** Client-initiated payment flow through orchestration engine
- **Asynchronous:** Gateway-initiated webhook ingestion pipeline
- Both converge on the same transaction record in PostgreSQL

Technology Stack



Backend

Node.js + NestJS — modular, typed architecture



Database

PostgreSQL — ACID source of truth



Cache

Redis — idempotency keys, health scores, circuit breaker state



Queue

RabbitMQ / Celery — webhook reconciliation, retries



Gateway SDKs

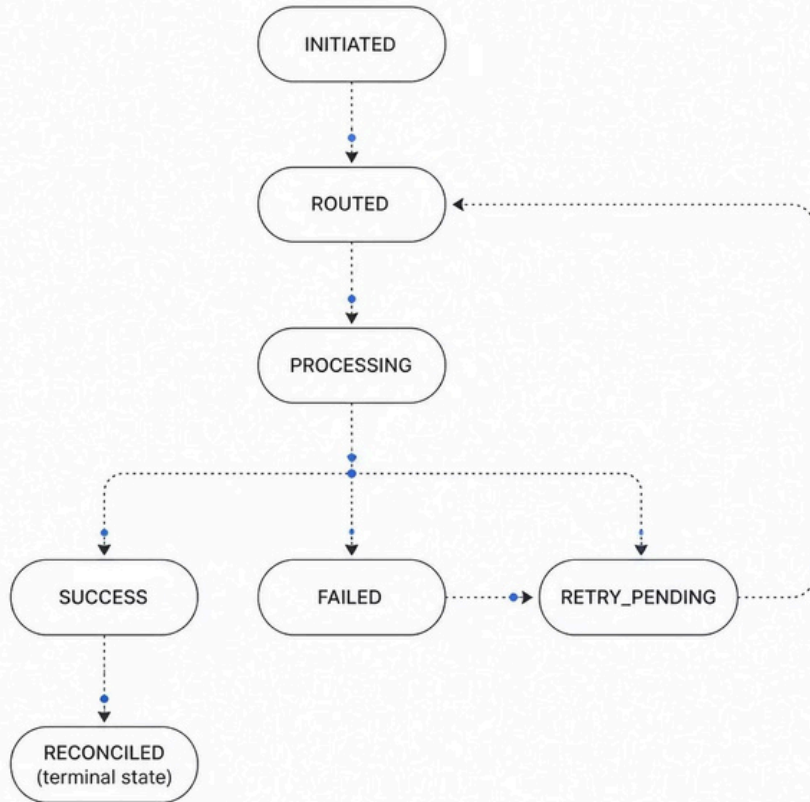
Stripe, Razorpay, PayU — test-mode sandboxes



Infrastructure

Docker Compose — local service orchestration

Transaction State Machine

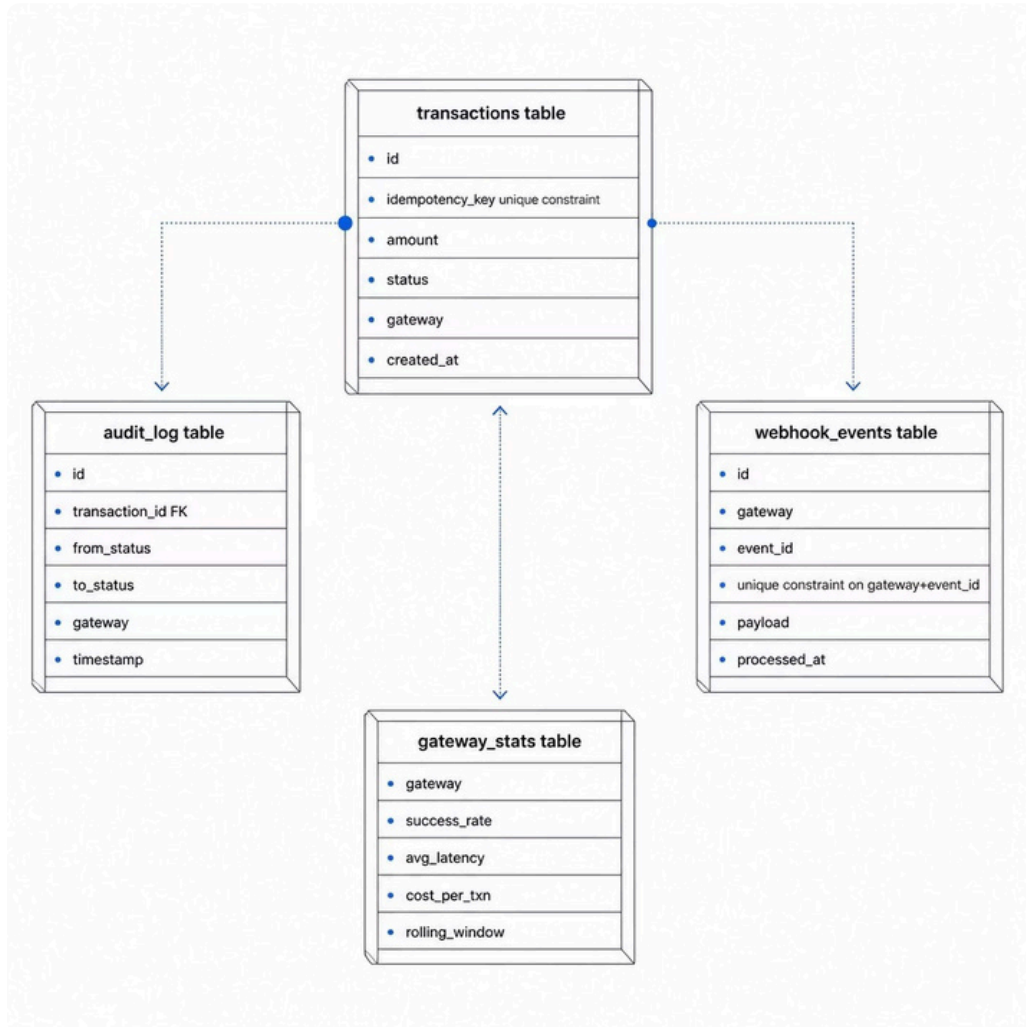


State Transition Rules

- Only the state machine mutates transaction state
- Every transition logged to immutable `audit_log`
- Explicit allow-list prevents backward moves
- **RECONCILED** is terminal — no late webhook can revert it

📌 **Key invariant:** A late webhook cannot move a **RECONCILED** transaction back to **PROCESSING**.

Database Schema



Two Constraints That Do the Real Work

Idempotency Key

Unique constraint on transactions.idempotency_key — prevents duplicate charges at the database level

Webhook Dedup

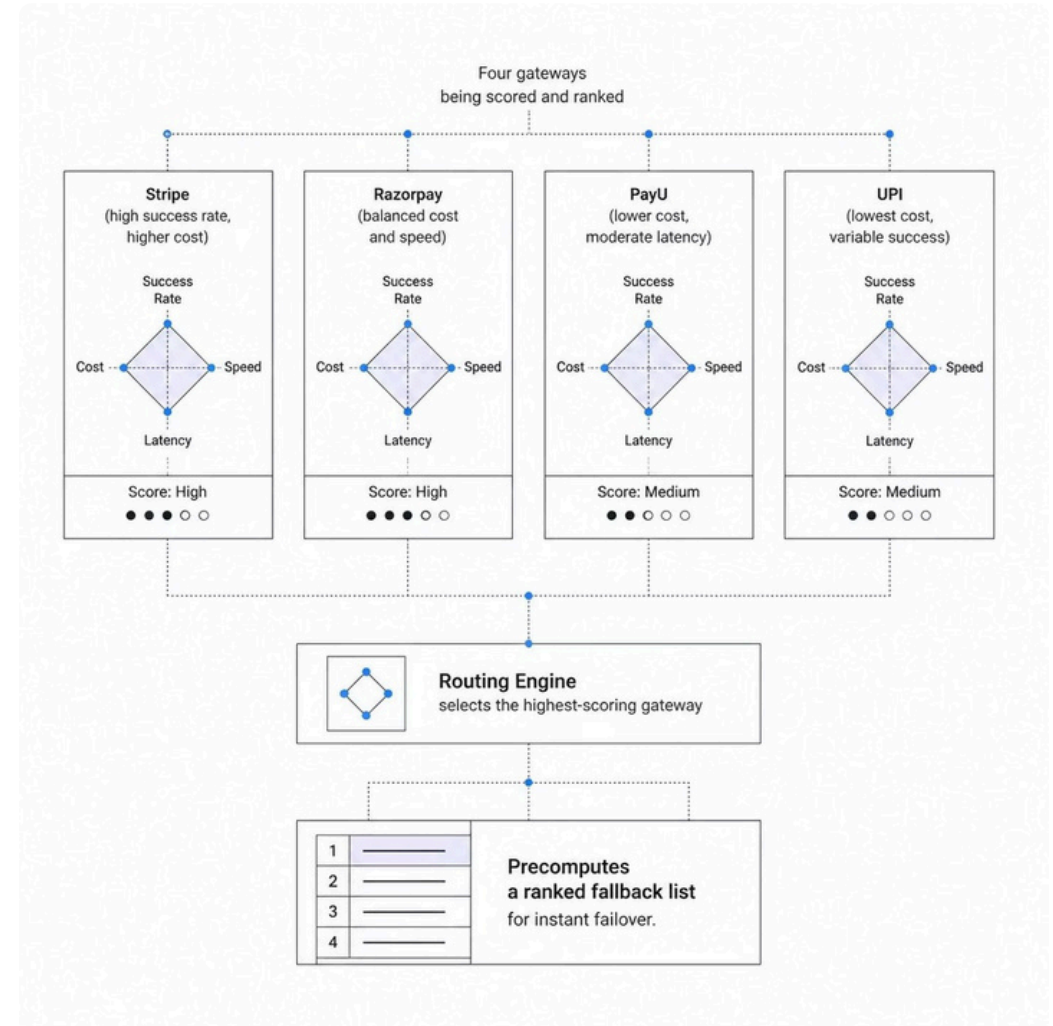
Unique constraint on (gateway, event_id) in webhook_events — gateways retry aggressively, this absorbs duplicates

Intelligent Routing Algorithm

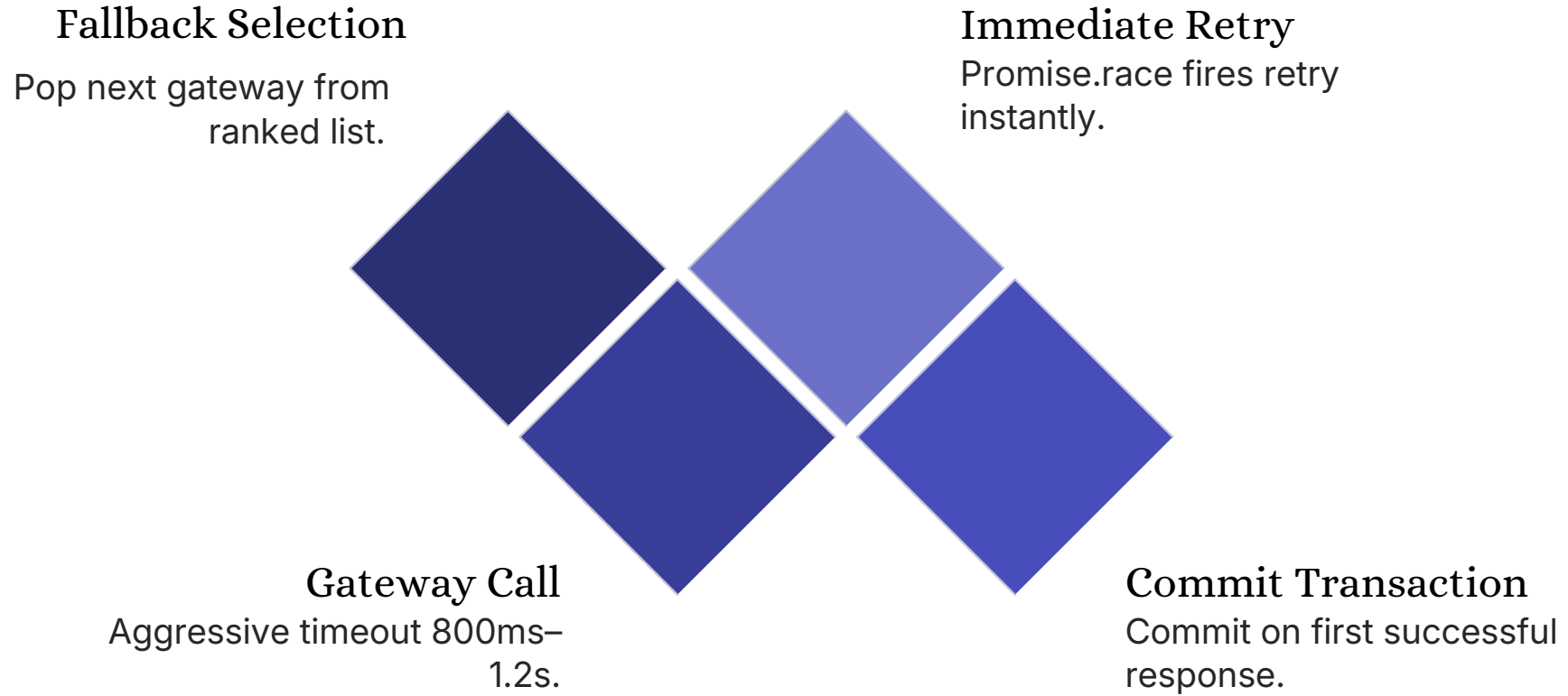
Gateway Scoring Formula

$$\text{score}(g) = w_s \cdot \text{success_rate} + w_c \cdot (1/\text{cost}) + w_l \cdot (1/\text{latency}) + w_h \cdot \text{health_multiplier}$$

- Rolling window computed per gateway
- Weights are configurable per deployment
- Ranked fallback list precomputed per transaction
- Gateway switch requires zero DB round-trip mid-failure



Failover & Circuit Breaker



Failover completes within the 2-second SLA via aggressive timeouts and a precomputed fallback list. The circuit breaker prevents cascading failures by isolating unhealthy gateways.

Idempotency & Webhook Ingestion

Dual-Layer Idempotency

- **Fast path:** Redis SETNX lock — O(1) duplicate detection
- **Durable backstop:** Postgres unique constraint
- Duplicate request returns cached transaction — no double-charge

Webhook Ingestion Pipeline

01

Receive & Verify Signature

02

Insert with Unique Constraint

03

Ack 200 Immediately

04

Push to Queue

05

Async Reconciler Updates State

-  Fast-ack + async processing is a deliberate design choice — gateways retry aggressively.

Audit Trail

Append-only forensic record of every routing decision and state transition.

Timestamp	From State	To State	Gateway	Note
10:00:01.102	—	INITIATED	—	Client request received
10:00:01.118	INITIATED	ROUTED	Stripe	Score: 0.87 (rank 1)
10:00:01.240	ROUTED	PROCESSING	Stripe	Gateway call sent
10:00:02.041	PROCESSING	FAILED	Stripe	Timeout after 800ms
10:00:02.043	FAILED	RETRY_PENDING	—	Fallback triggered
10:00:02.055	RETRY_PENDING	ROUTED	Razorpay	Score: 0.81 (rank 2)
10:00:02.190	ROUTED	PROCESSING	Razorpay	Gateway call sent
10:00:02.891	PROCESSING	SUCCESS	Razorpay	Payment confirmed
10:00:03.104	SUCCESS	RECONCILED	Razorpay	Webhook verified